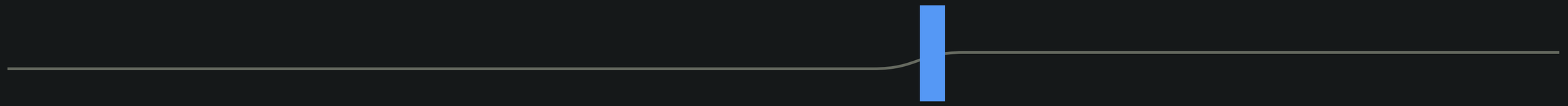
BEYOND AI CODING

The New Bottlenecks of Software Delivery



Lucas Mazei

Engineering Manager at Memed

A TALK ABOUT FINISHING



ABOUT THE SPEAKER

Lucas Mazei

Engineering Manager at Memed

A background in software engineering, product, and technology leadership.

Previously at V4 Company · Based in Brazil

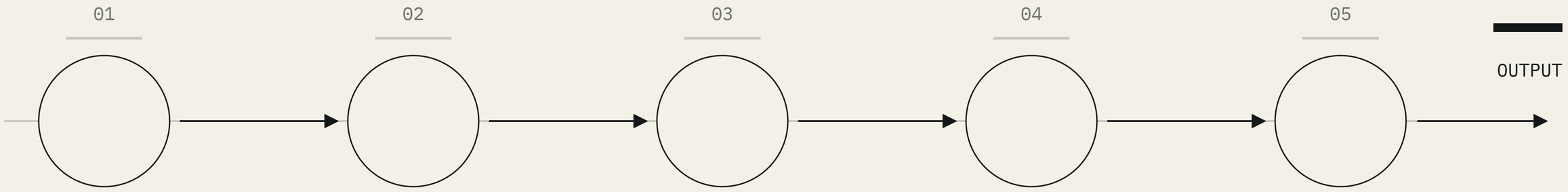
THE QUESTION THAT DRIVES THIS TALK

How do we help useful work move through the whole delivery system?

Five stages. One system.

EMPTY PIPELINE

CAPACITY / COMPARABLE UNITS PER DAY



Implement	Review	Validate	Release	Production
4 / day capacity	3 / day capacity	4 / day capacity	4 / day capacity	4 / day capacity
0 in service	0 in service	0 in service	0 in service	0 in service

0 in service · 0 waiting · 0 in transit

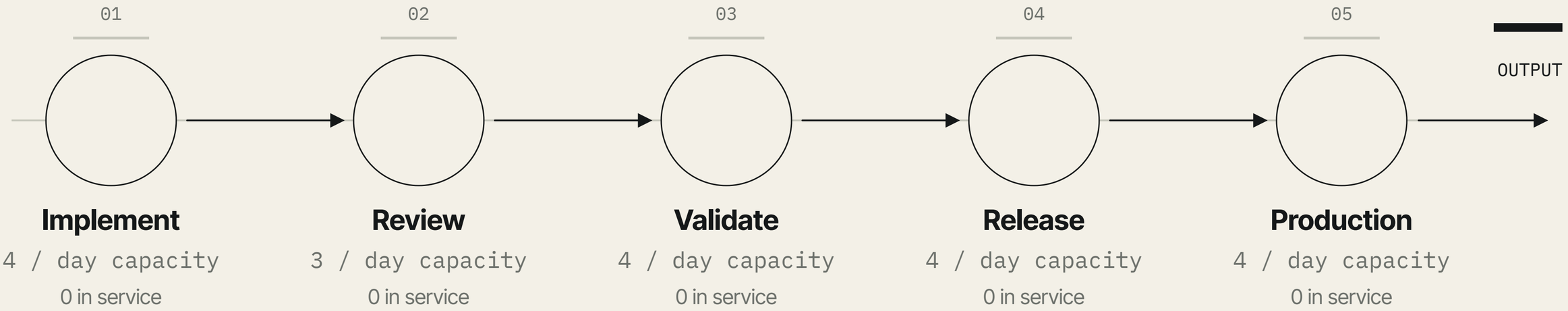
Illustrative capacities · each ticket = 1 unit

Five stages. One system.

EMPTY PIPELINE

CAPACITY / COMPARABLE UNITS PER DAY

What can the whole system finish per day?



0 in service | 0 waiting | 0 in transit

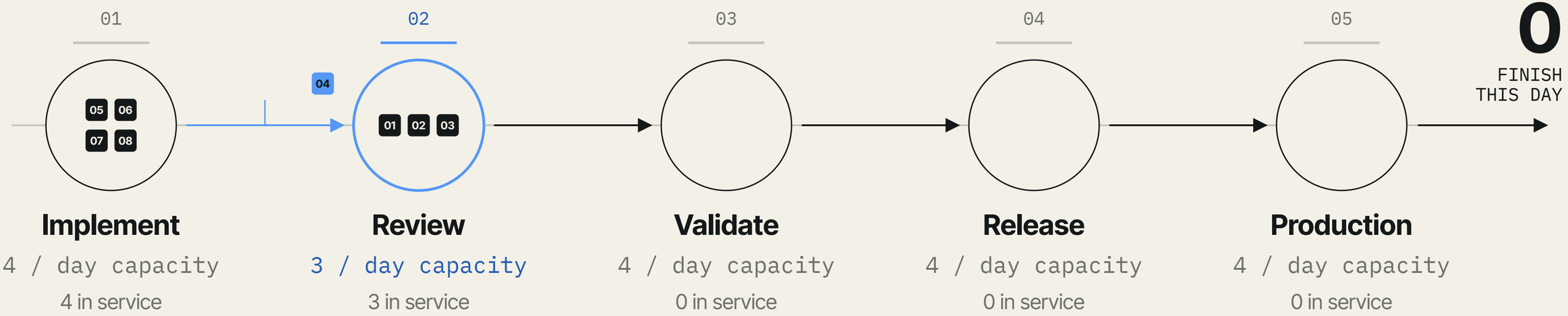
Illustrative capacities · each ticket = 1 unit

Four arrive. Three pass.

DURING DAY 2 · PIPELINE WARMING

4 arrive − 3 pass = +1 waiting / day

1
WAITING



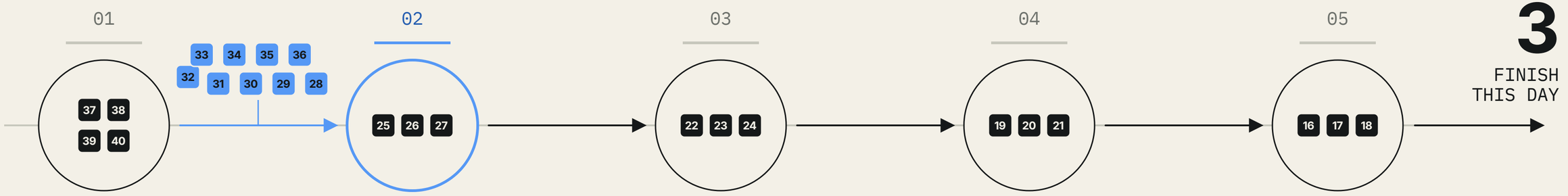
7 in service · 1 waiting · 0 in transit · 0 completed · 8 admitted · IDs retained

Four arrive. Three pass.

DURING DAY 10 · PIPELINE FLOWING

4 arrive − 3 pass = +1 waiting / day

9
WAITING



Implement	Review	Validate	Release	Production
4 / day capacity	3 / day capacity	4 / day capacity	4 / day capacity	4 / day capacity
4 in service	3 in service	3 in service	3 in service	3 in service

16 in service · 9 waiting · 0 in transit · 15 completed · 40 admitted · IDs retained

If coding gets much faster, what changes?

CODING



DELIVERY

?

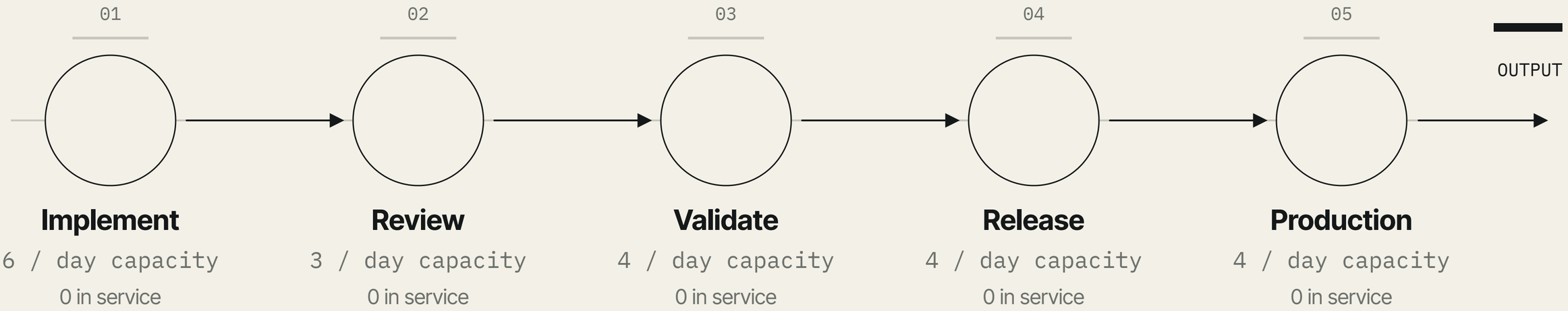


Where does the extra work go?

Now accelerate coding.

EMPTY PIPELINE

CAPACITY / CANDIDATE PRs PER DAY



0 in service | 0 waiting | 0 in transit

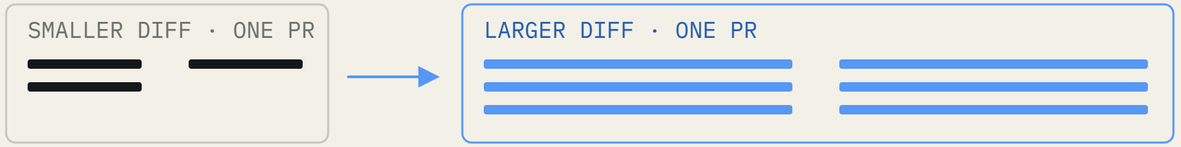
Illustrative capacities · each ticket = 1 unit

More changes inside each PR.

EMPTY PIPELINE

ILLUSTRATIVE: LARGER PRs TAKE MORE REVIEW WORK

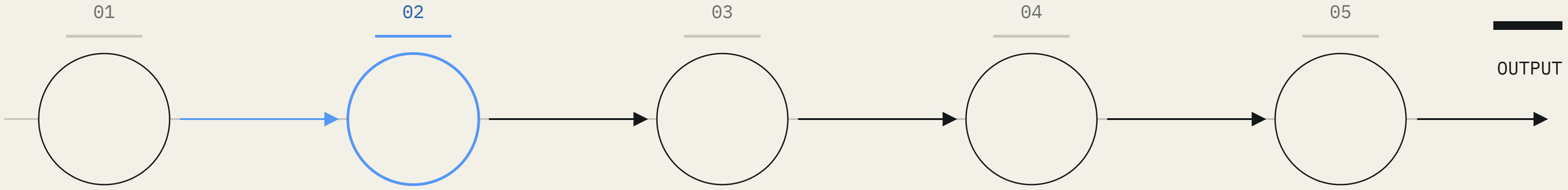
More edits, files and interactions.



Review: 3 → 2 PRs / day

Same people. More work per PR.

SCHEMATIC DIFFS · HYPOTHETICAL EFFECT



Implement	Review	Validate	Release	Production
6 / day capacity	2 / day capacity	4 / day capacity	4 / day capacity	4 / day capacity
0 in service	0 in service	0 in service	0 in service	0 in service

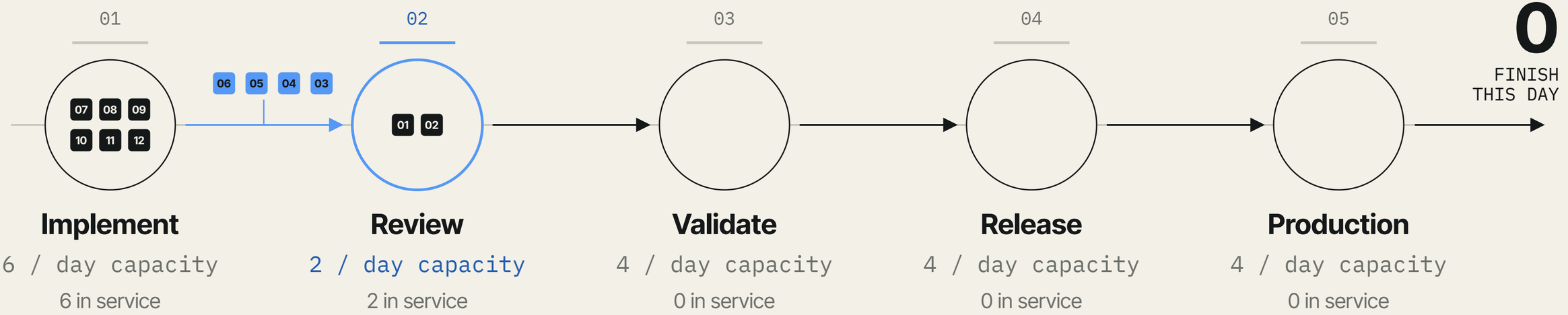
0 in service · 0 waiting · 0 in transit · Illustrative capacities · each ticket = 1 unit

Six arrive. Two pass.

DURING DAY 2 · PIPELINE WARMING

6 arrive − 2 pass = +4 waiting / day

4
WAITING



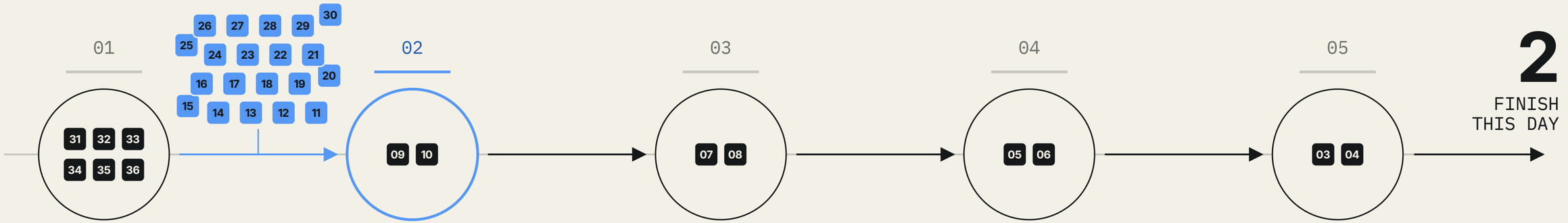
8 in service · 4 waiting · 0 in transit · 0 completed · 12 admitted · IDs retained

Six arrive. Two pass.

DURING DAY 6 · PIPELINE FLOWING

6 arrive − 2 pass = +4 waiting / day

20
WAITING



Implement	Review	Validate	Release	Production
6 / day capacity	2 / day capacity	4 / day capacity	4 / day capacity	4 / day capacity
6 in service	2 in service	2 in service	2 in service	2 in service

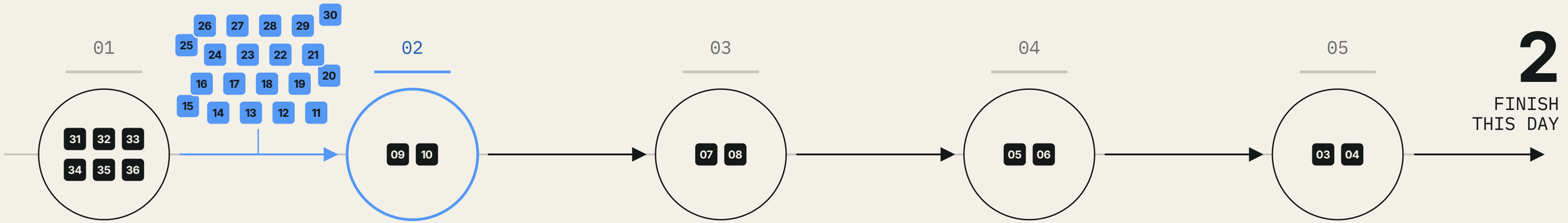
14 in service · 20 waiting · 0 in transit · 2 completed · 36 admitted · IDs retained

What if we just approve faster?

DURING DAY 6 · PIPELINE FLOWING

6 arrive − 2 pass = +4 waiting / day

20
WAITING



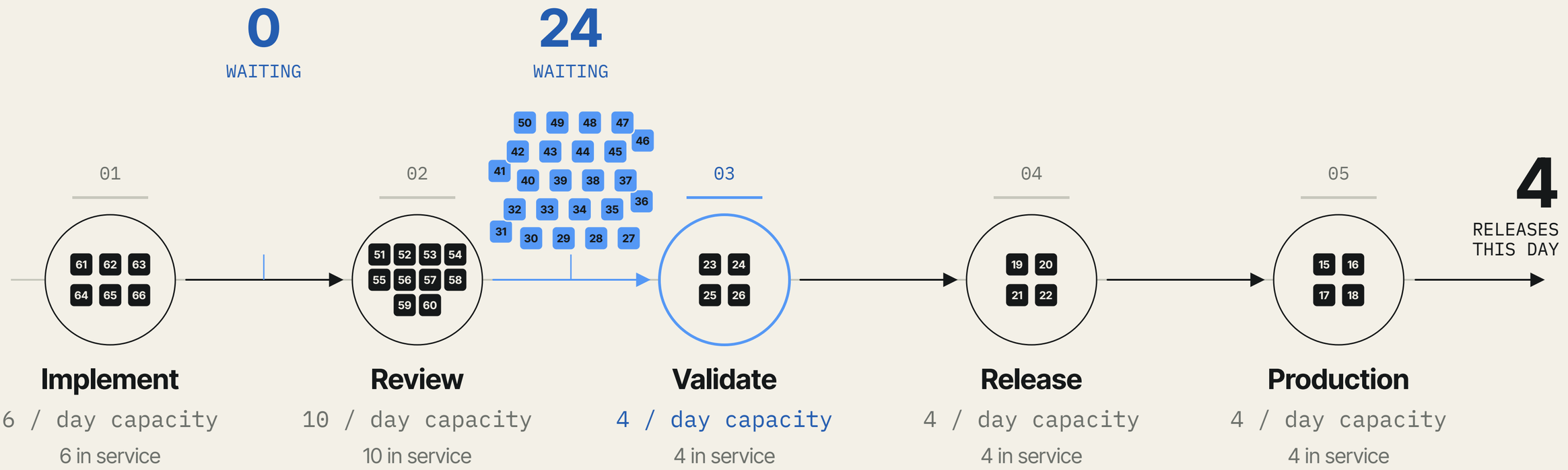
Implement	Review	Validate	Release	Production
6 / day capacity	2 / day capacity	4 / day capacity	4 / day capacity	4 / day capacity
6 in service	2 in service	2 in service	2 in service	2 in service

14 in service · 20 waiting · 0 in transit · 2 completed · 36 admitted · IDs retained

What if we just approve faster?

DURING DAY 11 · PIPELINE FLOWING

10 careless approvals / day · 6 arrivals / day

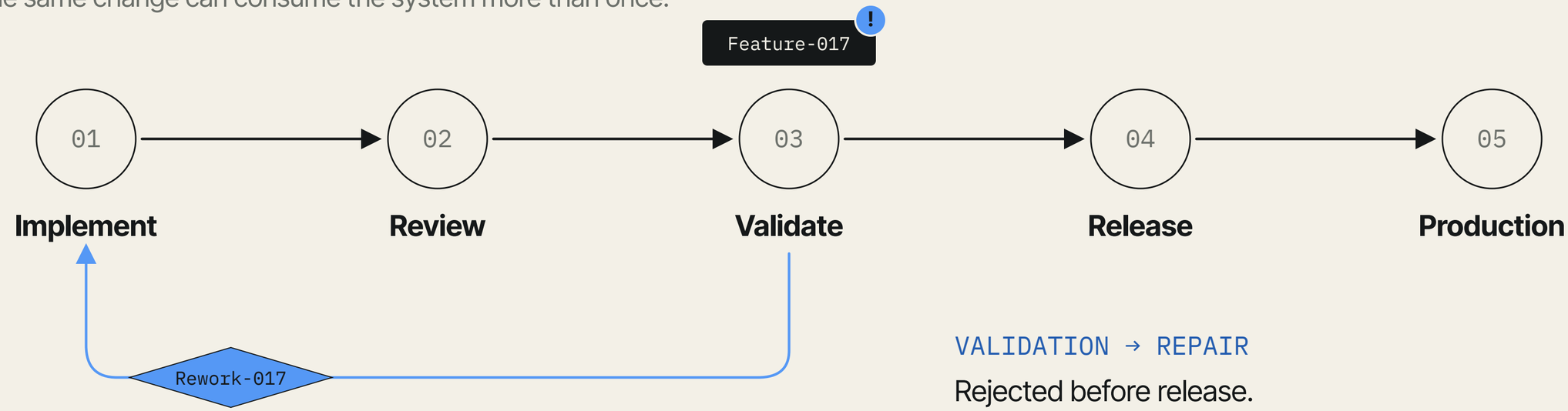


28 in service · 24 waiting · 0 in transit · 14 completed · 66 admitted · IDs retained

Same FIFO work · approval capacity changes on day 7 · releases do not establish useful outcomes

Follow the work that comes back.

The same change can consume the system more than once.



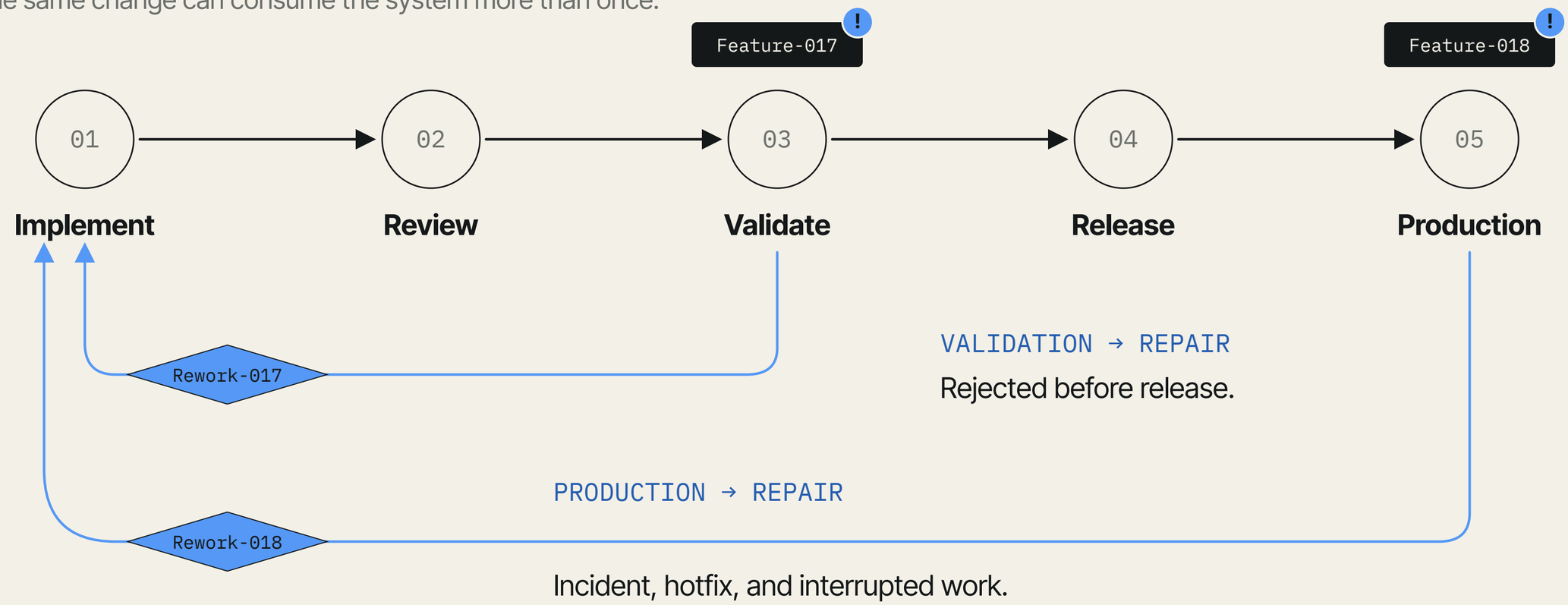
VALIDATION → REPAIR

Rejected before release.

! = defect discovered · diamonds = new repair work linked to an original change

Follow the work that comes back.

The same change can consume the system more than once.



! = defect discovered · diamonds = new repair work linked to an original change

Six approvals are not six outcomes.

SAME PEOPLE · LESS ATTENTION PER CHANGE

2

CAREFUL REVIEWS / DAY

ATTENTION PER CHANGE

The original careful-review rate.

Compare the work, not just the counter.

LESS ATTENTION CAN INCREASE THE APPROVAL COUNTER



Approval is an event. Safe delivery is an outcome.

After the queue drains, arrivals limit approvals to six/day · downstream capacity remains four/day

Six approvals are not six outcomes.

SAME PEOPLE · LESS ATTENTION PER CHANGE

2
CAREFUL REVIEWS / DAY

→

6
CARELESS APPROVALS / DAY

Looks like capacity.
What changed was
the depth of review.

ATTENTION PER CHANGE

After the backlog clears: six in, six approved.

How many were delivered safely?

10 / DAY APPROVAL CAPACITY · 6 / DAY ARRIVALS



Approval is an event. Safe delivery is an outcome.

After the queue drains, arrivals limit approvals to six/day · downstream capacity remains four/day

Start the loop with quality gates.

Check for regressions before asking the AI reviewer.

01 / DETERMINISTIC

Quality gates

Unused code · duplication
Coverage versus base
Required E2E flows

SAME BASE + PINNED RULES

Defined regressions, not proof of all quality · no lowered thresholds, weakened assertions, or stale evidence

Start the loop with quality gates.

Check for regressions before asking the AI reviewer.

01 / DETERMINISTIC

Quality gates

Unused code · duplication
Coverage versus base
Required E2E flows

PASS

02 / CONTEXTUAL

AI reviewer

Behavior · context · risk
Findings with evidence
Review the current revision

FINDINGS ADDRESSED

03 / JUDGMENT

Human

Focused diff
Evidence + risk
Open decisions

SAME BASE + PINNED RULES

GATE FAILURES

AI FINDINGS

One repair agent

NEW REVISION → RESTART GATES → RERUN AI REVIEW

Example budget: 2 repairs or 10 minutes. Then hand off.

Defined regressions, not proof of all quality · no lowered thresholds, weakened assertions, or stale evidence

Automate the mechanical decisions.

Scaffold a new resource. Add the behavior. Enforce the team's rules.

NESTJS CLI · NEW RESOURCE

```
$ nest g resource orders
```

REST API · CRUD entry points: Yes

orders.module.ts
orders.controller.ts
orders.service.ts
dto/create-order.dto.ts
dto/update-order.dto.ts
orders.service.spec.ts

SELECTED FILES · SERVICE + TEST SKELETONS

TEAM CHECKS / SEPARATE FROM CLI

Lint + architecture rules
Required CI checks
Same rules for people and agents

The generator creates the structure. The export behavior still needs work.

Documented NestJS scaffolding · selected output · team rules are configured separately · no flow test generated

Automate the mechanical decisions.

Scaffold a new resource. Add the behavior. Enforce the team's rules.

NESTJS CLI · NEW RESOURCE

```
$ nest g resource orders
REST API · CRUD entry points: Yes

orders.module.ts
orders.controller.ts
orders.service.ts
dto/create-order.dto.ts
dto/update-order.dto.ts
orders.service.spec.ts
```

SELECTED FILES · SERVICE + TEST SKELETONS

TEAM CHECKS / SEPARATE FROM CLI

Lint + architecture rules
Required CI checks
Same rules for people and agents

HUMAN ATTENTION

Permissions. Tenant isolation.
Field meaning. Data exposure.
The business behavior inside the diff.

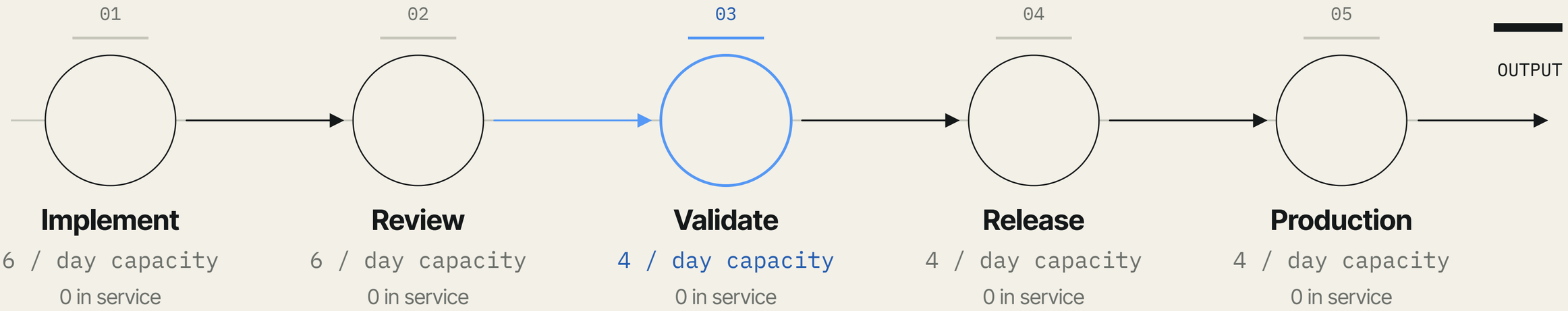
The generator creates the structure. The export behavior still needs work.

Documented NestJS scaffolding · selected output · team rules are configured separately · no flow test generated

Now six means review capability.

EMPTY PIPELINE

6 reviewed − 4 validated = +2 waiting / day



0 in service 0 waiting 0 in transit

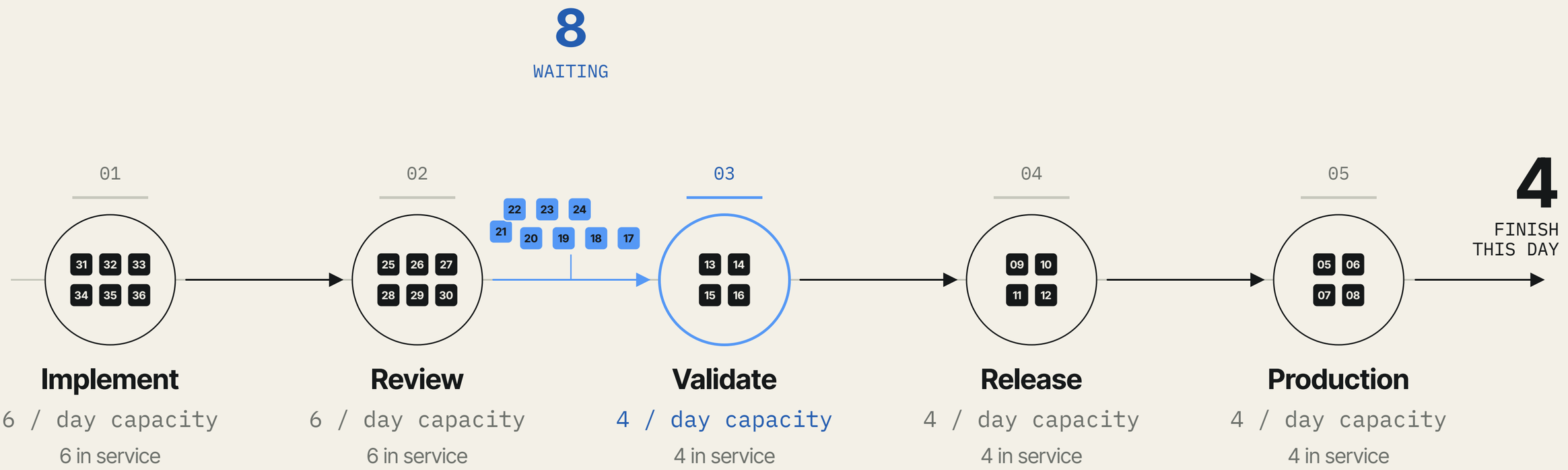
Illustrative capacities · each ticket = 1 unit

Quality held · validation is the first of three stages sharing the 4/day ceiling.

Now six means review capability.

DURING DAY 6 · PIPELINE FLOWING

6 reviewed − 4 validated = +2 waiting / day



24 in service | 8 waiting | 0 in transit | 4 completed · 36 admitted · IDs retained

Quality held · validation is the first of three stages sharing the 4/day ceiling.

Explore the validation workflow.

Write the verification alongside the implementation.

EXPLORE WITH COMPUTER USE

An agent uses the product.
It clicks, types, observes,
and records the unexpected.

EXAMPLE: CLAUDE COMPUTER USE

Screenshots + actions + observations

Proposed validation workflow · review changes to test expectations; a healer must not silently weaken them

Make the check repeatable.

Write the verification alongside the implementation.

EXPLORE WITH COMPUTER USE

An agent uses the product.
It clicks, types, observes,
and records the unexpected.

EXAMPLE: CLAUDE COMPUTER USE

Screenshots + actions + observations

REPEAT WITH EXECUTABLE TESTS

Playwright Test Agents
Plan → generate → run
Keep assertions explicit.

Seeded state. Known identities.

Trace and results tied to the revision.

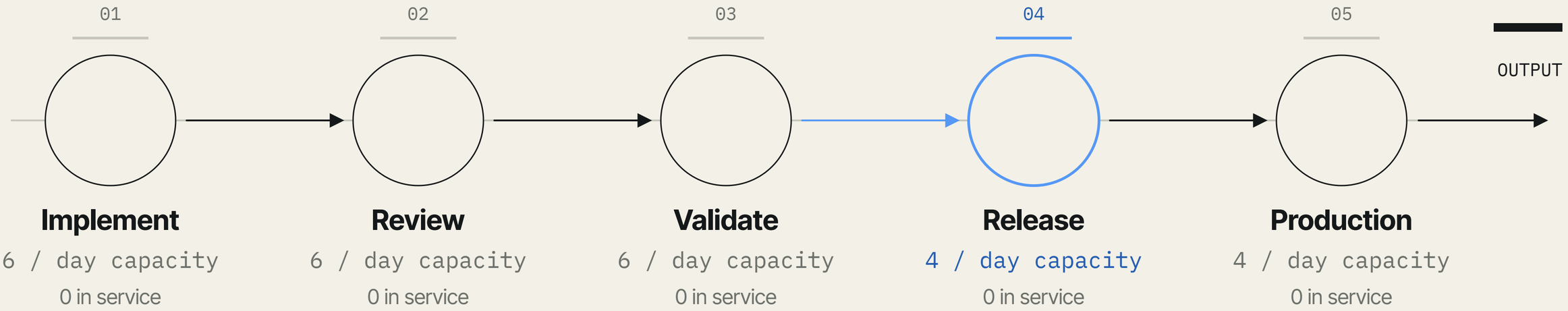
The agent can explore. The team defines what correct means.

Proposed validation workflow · review changes to test expectations; a healer must not silently weaken them

The next limit is release.

EMPTY PIPELINE

6 validated − 4 released = +2 waiting / day



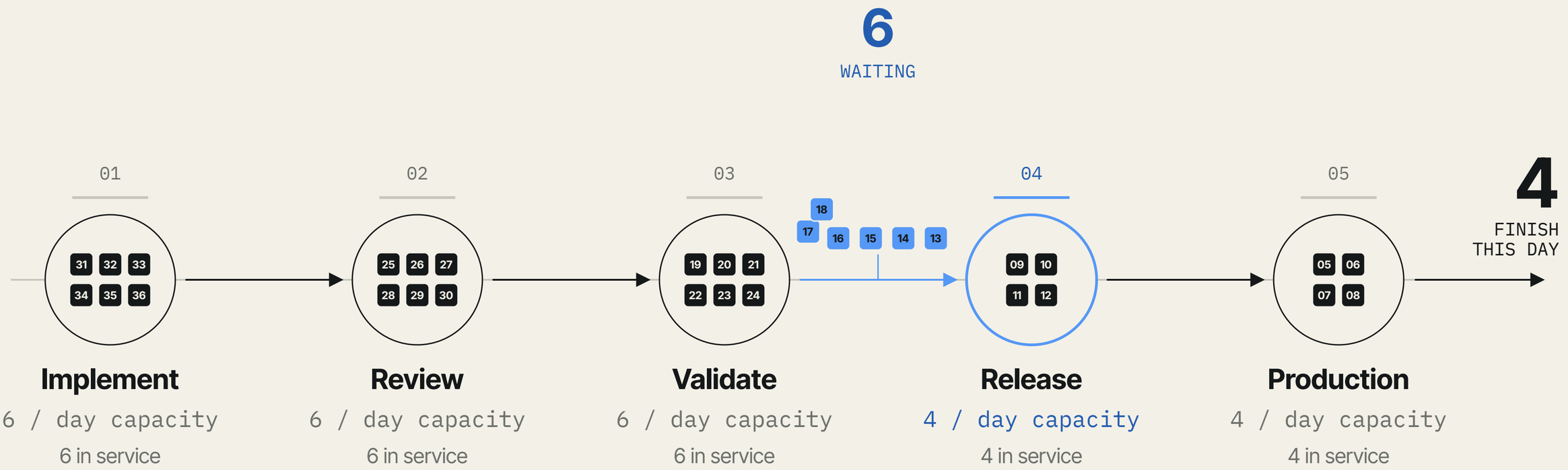
0 in service 0 waiting 0 in transit Illustrative capacities · each ticket = 1 unit

Quality held · release and production share the 4/day ceiling; improve the release path together.

The next limit is release.

DURING DAY 6 · PIPELINE FLOWING

6 validated − 4 released = +2 waiting / day



26 in service · 6 waiting · 0 in transit · 4 completed · 36 admitted · IDs retained

Quality held · release and production share the 4/day ceiling; improve the release path together.

Branch policy changes the batch.

Choose the workflow that keeps your release unit small and understandable.

Short-lived branch → main

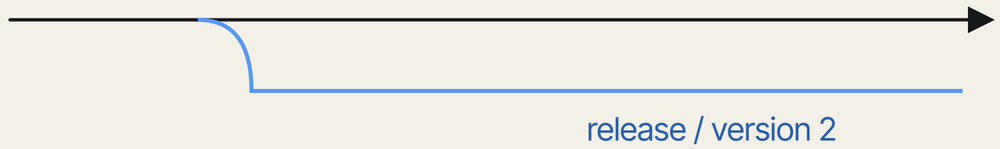
GITHUB FLOW / TRUNK-BASED PRACTICES



Integrate frequently. Run CI on the merge.
Use flags to separate deployment from exposure.

Versioned release branches

GITFLOW-STYLE RELEASE COORDINATION



Useful for explicit version/support needs.
Price in merges, backports, and waiting.

A branch strategy does not replace a reliable delivery pipeline.

Workflow tradeoffs, not a universal prescription · release branches should solve an observed coordination need

Measure useful delivery and quality.

01 **Useful delivery**

Comparable outcomes that reached the user

Lead time · work age · distinct completed changes

02 **Quality**

Failures and work that returned

Escaped defects · change failures · repair effort

More approvals can be useful. They cannot be the definition of success.

Count AI cost per useful outcome.

01	**Useful delivery**	Comparable outcomes that reached the user Lead time · work age · distinct completed changes
02	**Quality**	Failures and work that returned Escaped defects · change failures · repair effort
03	**AI economics**	Spend per useful delivered outcome Implementation + review + repair + validation usage

More approvals can be useful. They cannot be the definition of success.

New bottlenecks. Same discipline.

Beyond AI coding: where does your delivery flow get stuck?

Manager. Developer. Platform engineer. This is our work.

Lucas Mazei · Engineering Manager at Memed

Thank you.

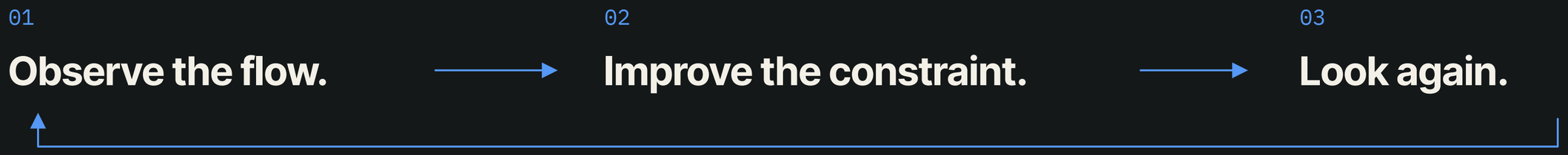
Follow the flow.
Find the constraint.

Beyond AI coding: where does your delivery flow get stuck?

Manager. Developer. Platform engineer. This is our work.

When this step stops limiting delivery, find the next constraint.

It may be elsewhere in the system. Look again.

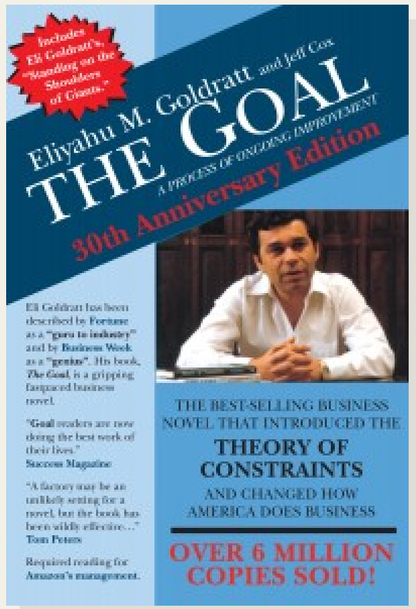


Lucas Mazei · Engineering Manager at Memed

Thank you.

Two books behind the talk.

Constraints. Visibility. Ongoing improvement.

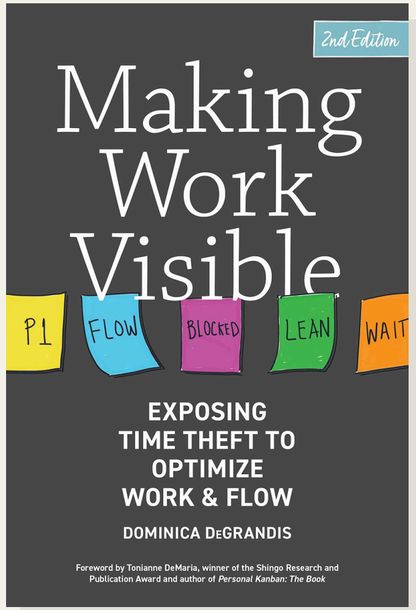


The Goal

Eliyahu M. Goldratt
and Jeff Cox

30th anniversary edition
North River Press

Find the constraint.
Improve the system.



Making Work Visible

Dominica DeGrandis

Second edition
IT Revolution

Make work visible.
See what holds it up.

The Goal — Eliyahu M. Goldratt & Jeff Cox

Making Work Visible — Dominica DeGrandis